

# 自动调整学习率 (Adaptive Learning Rate)

李宏毅《机器学习》2025 · 类神经网络训练不起来怎么办  
(三)



基于李宏毅老师课程整理

2026 年 6 月 5 日

## 视频信息

频道：李宏毅深度学习课堂 (Bilibili 搬运)

时长：约 38 分钟

链接：<https://www.bilibili.com/video/BV1TAtwzTE1S?p=6>

# 目录

|     |                                            |    |
|-----|--------------------------------------------|----|
| 1   | 引言：Loss 不再下降，真的是卡在 critical point 吗？       | 3  |
| 2   | 固定学习率的两难：凸面上的震荡与停滞                         | 4  |
| 3   | 客制化学习率与 Adagrad                            | 5  |
| 3.1 | 把固定步长换成“参数专属”步长                            | 6  |
| 3.2 | Root Mean Square：最经典的 $\sigma$ 算法（Adagrad） | 7  |
| 4   | RMSProp：让 $\sigma$ 跟得上 error surface 的变化   | 10 |
| 5   | Adam：RMSProp + Momentum                    | 13 |
| 6   | 学习率调度：Decay 与 Warm Up                      | 14 |
| 6.1 | 动机：Adagrad 会“喷出去”再自我修正                     | 14 |
| 6.2 | Learning Rate Decay：越接近终点，步子越小             | 15 |
| 6.3 | Warm Up：先升后降的反直觉技巧                         | 16 |
| 7   | 优化的全景总结（Summary of Optimization）           | 17 |
| 8   | 总结与延伸                                      | 18 |
| 8.1 | 核心要点提炼                                     | 18 |
| 8.2 | 本节关键概念清单                                   | 19 |
| 8.3 | 承上启下                                       | 19 |

# 1 引言: Loss 不再下降, 真的是卡在 critical point 吗?

上一节我们得到一个安慰人的结论: 训练停滞时, 多半不是卡在 local minima, 而是卡在 saddle point, 而高维空间里 saddle point 周围几乎总有路可走。听起来只要有耐心, loss 终究能继续往下掉。

但这一节老师要先泼一盆冷水: 当你的 training loss 卡住、不再下降时, 很多时候它根本没有走到 critical point 附近。也就是说, “gradient 趋近于零” 常常不是训练失败的真凶。

[https://docs.google.com/presentation/d/1siUFXARYRpNiMeSRwgFbt7mZVjkMPhR5od09w0Z8xa/edit#slide=id.g3532c09be1\\_0\\_382](https://docs.google.com/presentation/d/1siUFXARYRpNiMeSRwgFbt7mZVjkMPhR5od09w0Z8xa/edit#slide=id.g3532c09be1_0_382)

## Training stuck $\neq$ Small Gradient

- People believe training stuck because the parameters are around a critical point ...

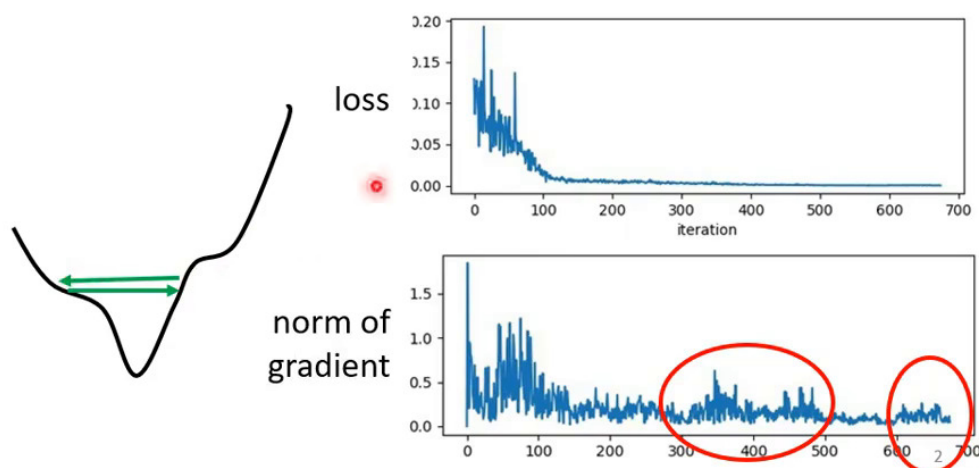


图 1: Loss 卡在高处不动, 人们直觉以为是走到了 gradient 为零的 critical point。但把 gradient 的大小 (norm) 画出来看, 会发现当 loss 不再下降时, gradient 的 norm 并没有真的变小——它仍然很大, 只是在某处反复“弹来弹去”。可见训练停滞与“gradient = 0”往往是两回事。<sup>1</sup>

### 一个常见的误判: “train 不下去” $\neq$ “走到 critical point”

很多人一看到 loss 平在那里不动, 就断定“一定是卡在 local minima 或 saddle point”, 于是急着去算 Hessian、找逃生方向。但实测告诉我们: loss 卡住时, gradient 的大小常常还很可观。真正发生的, 往往是参数在 error surface 的“峡谷两壁”之间来回横跳, 每一步 gradient 都不小, loss 却始终下不去。问题的根源不在 critical point, 而在步伐 (learning rate) 没调好。

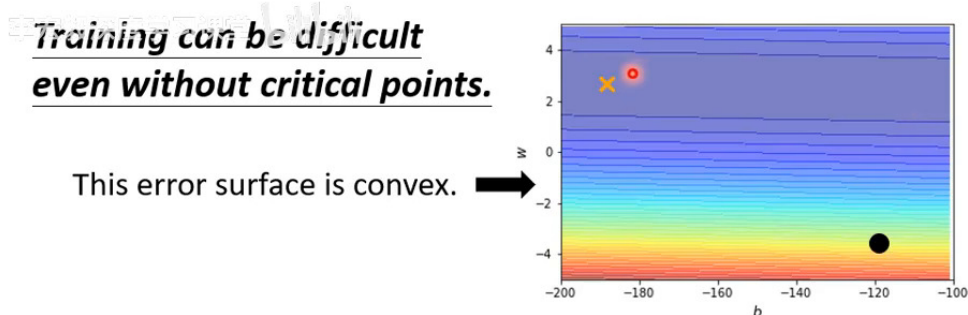
<sup>1</sup> 视频画面时间区间: 00:02:17–00:02:54。

### 在“山谷”两壁之间反复震荡

想象一条狭长的山谷（error surface 在某个方向很陡、另一个方向很平）。如果 learning rate 偏大，参数沿陡峭方向一步就冲过谷底、撞上对面山壁，下一步又被弹回来——如此左右横跳，loss 始终降不下去，而 gradient 一直很大。这类停滞，靠“逃离 critical point”的思路是治不好的，只能从学习率本身下手。这正是本节的主题：把固定的 learning rate 换成会自动调整的 learning rate。

## 2 固定学习率的两难：凸面上的震荡与停滞

为了把问题看得最清楚，老师特意构造了一个极简、而且没有任何 critical point 的 error surface——一个凸函数（convex）误差曲面，等高线是一组又窄又长的椭圆。它在某个方向非常陡峭，在与之垂直的方向却非常平缓。



4

图 2: 实验设定：一个凸的 error surface，只有两个参数（横轴  $b$ 、纵轴  $w$ ）。等高线呈狭长椭圆——纵向（ $w$  方向）很陡，横向（ $b$  方向）很平。叉号  $\times$  标出的是 loss 最低点（终点），黑点是参数的起点。这个曲面没有 local minima、也没有 saddle point，照理说梯度下降“闭着眼”都该走到终点。<sup>2</sup>

可即便在这么“友善”的曲面上，固定 learning rate 的梯度下降也会陷入两难。

<sup>2</sup>视频画面时间区间：00:05:07–00:06:30。

Training can be difficult even without critical points.

This error surface is convex. →

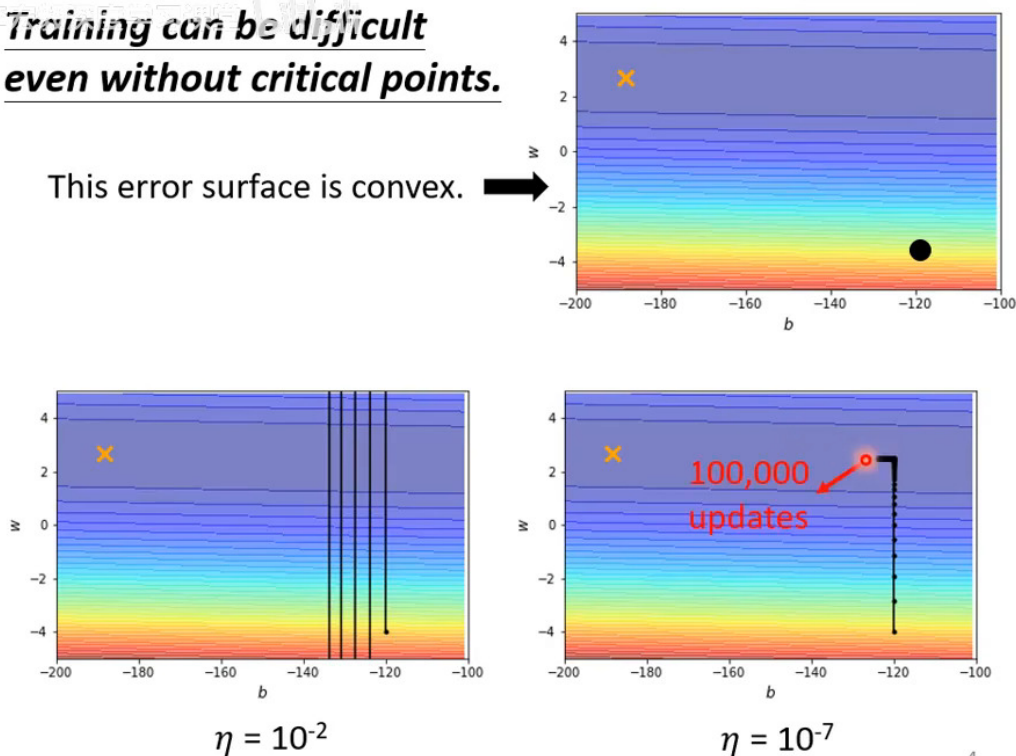


图 3: 两种固定学习率的实验对比。左 ( $\eta = 10^{-2}$ , 较大): 参数沿陡峭的  $w$  方向一步冲过头、撞上对面山壁, 于是在两壁之间剧烈上下震荡, 始终无法转入平缓的  $b$  方向走向终点。右 ( $\eta = 10^{-7}$ , 极小): 纵向终于不震荡了, 乖乖滑到谷底; 可一旦要沿平缓的  $b$  方向前进, 因步子太小走得无比缓慢, 几十万步也到不了叉号终点。结论一句话: *Learning rate cannot be one-size-fits-all* (学习率不能一视同仁)。<sup>3</sup>

#### 固定 $\eta$ 的两难: 大也不行, 小也不行

在这个“一边陡、一边平”的曲面上, 单一的学习率无论取多大都顾此失彼:

- $\eta$  偏大: 能推动平缓方向, 却让陡峭方向剧烈震荡 (在山壁间横跳, loss 降不下去)。
- $\eta$  偏小: 陡峭方向稳了, 平缓方向却慢到几乎不动, 永远走不到终点。

根本症结在于: 陡峭的方向需要小步、平缓的方向需要大步, 而固定学习率对所有方向一律同步长。我们需要给不同的参数 (方向)、甚至不同的时刻, 配上不同的学习率。

### 3 客制化学习率与 Adagrad

既然“一个  $\eta$  走天下”行不通, 自然的想法就是: 为每一个参数  $\theta_i$  量身定做一个属于它自己的学习率, 而且这个学习率还要能随训练动态调整。

<sup>3</sup>视频画面时间区间: 00:07:03–00:08:35。

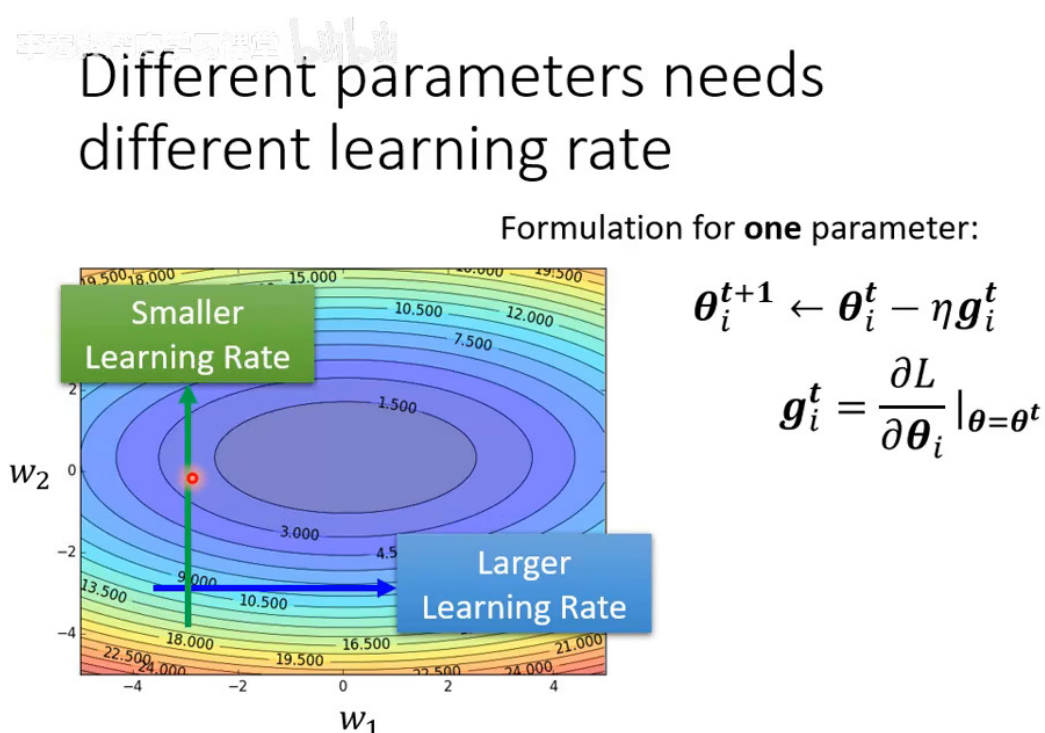
### 3.1 把固定步长换成“参数专属”步长

先回忆最朴素 (vanilla) 的梯度下降, 对第  $i$  个参数:

$$\theta_i^{t+1} \leftarrow \theta_i^t - \eta g_i^t, \quad g_i^t = \left. \frac{\partial L}{\partial \theta_i} \right|_{\theta=\theta^t}$$

- $\theta_i^t$ : 第  $i$  个参数在第  $t$  次迭代时的值 (上标  $t$  是**迭代时刻**, 不是次方)。
- $g_i^t$ : 第  $t$  步时 loss 对该参数的偏微分, 即 gradient 的第  $i$  个分量。
- $\eta$ : learning rate, 对所有参数**完全相同**、且**不随时间变**——这正是问题所在。

改造的办法, 是把分母里塞进一个**随参数、随时刻而变**的量  $\sigma_i^t$ :



5

图 4: 客制化学习率的核心写法: 把更新式改成  $\theta_i^{t+1} \leftarrow \theta_i^t - \frac{\eta}{\sigma_i^t} g_i^t$ 。原来对所有参数共用的  $\eta$ , 现在被替换成**每个参数各自专属**、且**逐步变化**的有效步长  $\eta/\sigma_i^t$ 。问题就归结为: 这个 **parameter-dependent** 的  $\sigma_i^t$  该怎么算? <sup>4</sup>

<sup>4</sup> 视频画面时间区间: 00:10:16–00:12:12。

## 客制化学习率的设计原则

新的更新式为

$$\theta_i^{t+1} \leftarrow \theta_i^t - \frac{\eta}{\sigma_i^t} g_i^t.$$

其中  $\sigma_i^t$  带有下标  $i$  (因参数而异) 和上标  $t$  (因时刻而异)。我们希望它满足：

- 某方向一向平缓 (gradient 小)  $\Rightarrow$  让  $\sigma_i^t$  小  $\Rightarrow$  有效步长  $\eta/\sigma_i^t$  变大, 大胆迈步。
- 某方向一向陡峭 (gradient 大)  $\Rightarrow$  让  $\sigma_i^t$  大  $\Rightarrow$  有效步长 变小, 小心慢行。

一句话：用 gradient 自身的大小来反向调节步长。

### 3.2 Root Mean Square: 最经典的 $\sigma$ 算法 (Adagrad)

最常见的  $\sigma_i^t$  取法, 是历来所有 gradient 的“均方根” (Root Mean Square, RMS):

李宏毅机器学习课程 bilibili

Root Mean Square  $\theta_i^{t+1} \leftarrow \theta_i^t - \frac{\eta}{\sigma_i^t} g_i^t$

$$\begin{aligned} \theta_i^1 &\leftarrow \theta_i^0 - \frac{\eta}{\sigma_i^0} g_i^0 & \sigma_i^0 &= \sqrt{(g_i^0)^2} = |g_i^0| \\ \theta_i^2 &\leftarrow \theta_i^1 - \frac{\eta}{\sigma_i^1} g_i^1 & \sigma_i^1 &= \sqrt{\frac{1}{2} [(g_i^0)^2 + (g_i^1)^2]} \\ \theta_i^3 &\leftarrow \theta_i^2 - \frac{\eta}{\sigma_i^2} g_i^2 & \sigma_i^2 &= \sqrt{\frac{1}{3} [(g_i^0)^2 + (g_i^1)^2 + (g_i^2)^2]} \\ &\vdots & & \\ \theta_i^{t+1} &\leftarrow \theta_i^t - \frac{\eta}{\sigma_i^t} g_i^t \end{aligned}$$

6

图 5: Root Mean Square 的定义与逐步展开。 $\sigma_i^t$  取为“从第 0 步到第  $t$  步、该参数所有 gradient 的平方平均, 再开根号”。这个用 RMS 当分母的方法, 搭配梯度下降就是经典的 **Adagrad**。<sup>5</sup>

写成公式, 对第  $i$  个参数:

$$\sigma_i^t = \sqrt{\frac{1}{t+1} \sum_{j=0}^t (g_i^j)^2}$$

逐步展开, 便于看清它如何“累积”历史信息:

<sup>5</sup> 视频画面时间区间: 00:12:21–00:15:45。



$$\begin{aligned}
\sigma_i^0 &= \sqrt{(g_i^0)^2} = |g_i^0| \\
\sigma_i^1 &= \sqrt{\frac{1}{2} \left[ (g_i^0)^2 + (g_i^1)^2 \right]} \\
\sigma_i^2 &= \sqrt{\frac{1}{3} \left[ (g_i^0)^2 + (g_i^1)^2 + (g_i^2)^2 \right]} \\
&\vdots \\
\sigma_i^t &= \sqrt{\frac{1}{t+1} \sum_{j=0}^t (g_i^j)^2}
\end{aligned}$$

各符号含义：

- 求和指标  $j$  跑遍从开始到当前的每一个迭代时刻； $g_i^j$  是参数  $i$  在第  $j$  步的 gradient 分量。
- 取平方再平均，是为了只看 **gradient 的大小、不管正负方向**（正负平方后都为正，不会相互抵消）。
- 开根号让  $\sigma_i^t$  与 gradient 量纲一致，从而  $\eta/\sigma_i^t$  是一个合理的步长缩放。

### 为什么 Adagrad 真的有效

把这套机制放回那个“一边陡、一边平”的曲面看（见下图）：

- **陡峭方向**：历来 gradient 都大  $\Rightarrow$  RMS  $\sigma_i^t$  大  $\Rightarrow$  有效步长  $\eta/\sigma_i^t$  自动**缩小**，不再冲过头震荡。
- **平缓方向**：历来 gradient 都小  $\Rightarrow$   $\sigma_i^t$  小  $\Rightarrow$  有效步长自动**放大**，不再龟速爬行。

于是同一个  $\eta$ ，经过  $\sigma_i^t$  的“因地制宜”，就能让陡峭方向小步、平缓方向大步，化解上一节的两难。

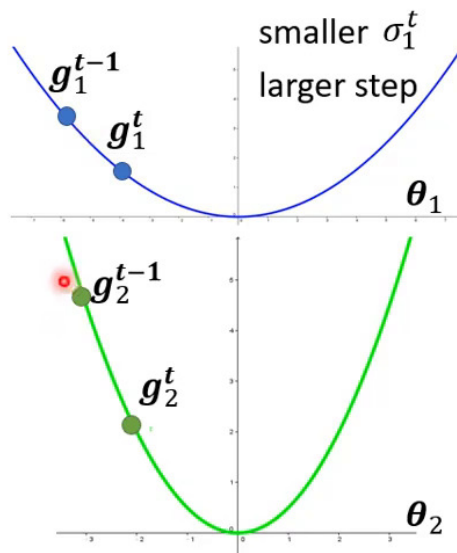


# Root Mean Square

$$\theta_i^{t+1} \leftarrow \theta_i^t - \frac{\eta}{\sigma_i^t} g_i^t$$

$$\sigma_i^t = \sqrt{\frac{1}{t+1} \sum_{i=0}^t (g_i^t)^2}$$

Used in **Adagrad**



7

图 6: Adagrad 何以奏效的图解。左侧“坡度小 (small gradient)”的参数，因历史梯度都小、 $\sigma$  小，得到**较大的学习率**；右侧“坡度大 (large gradient)”的参数，因历史梯度都大、 $\sigma$  大，得到**较小的学习率**。学习率就这样**随每个参数的陡缓自动配置**。<sup>6</sup>

## Adagrad 的盲点：只能“因方向制宜”，不能“因时制宜”

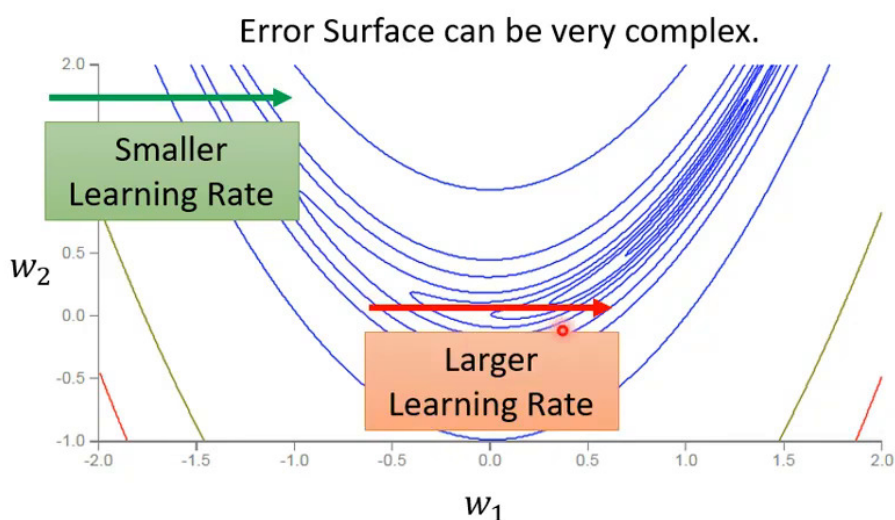
Adagrad 把整段历史的 gradient 一视同仁地平均，因此它假设同一个参数的陡缓是“一成不变”的。可现实中，同一个参数、同一个方向，在 error surface 的不同区段也可能时陡时缓。这时把远古的、早已不合时宜的 gradient 还一直算进平均里，反应就会迟钝。这正是下一节 RMSProp 要解决的问题。

<sup>6</sup>视频画面时间区间：00:15:45–00:17:41。

#### 4 RMSProp: 让 $\sigma$ 跟得上 error surface 的变化

手在键盘敲代码 bilibili

### Learning rate adapts dynamically



8

图 7: 即便是同一个参数、同一个方向, 也可能需要随时间变化的学习率。图中“新月形 (crescent)”的 error surface: 参数沿同一个方向前进时, 会先经过平缓段、再进入陡峭段。理想的做法是平缓时步子大、陡峭时步子小——可 Adagrad 用整段历史的平均  $\sigma$ , 对这种“中途变陡”的反应太慢。<sup>7</sup>

RMSProp 的改动很小却很关键: 不再对历史 gradient 一视同仁, 而是给“最近的” gradient 更大的话语权。

<sup>7</sup> 视频画面时间区间: 00:17:41–00:19:07。

$$\text{RMSProp} \quad \theta_i^{t+1} \leftarrow \theta_i^t - \frac{\eta}{\sigma_i^t} g_i^t$$

$$\theta_i^1 \leftarrow \theta_i^0 - \frac{\eta}{\sigma_i^0} g_i^0 \quad \sigma_i^0 = \sqrt{(g_i^0)^2} \quad 0 < \alpha < 1$$

$$\theta_i^2 \leftarrow \theta_i^1 - \frac{\eta}{\sigma_i^1} g_i^1 \quad \sigma_i^1 = \sqrt{\alpha(\sigma_i^0)^2 + (1 - \alpha)(g_i^1)^2}$$

9

图 8: RMSProp 的递归定义与逐步展开。第一步同样取  $\sigma_i^0 = |g_i^0|$ ; 此后每一步的  $\sigma_i^t$  都是“上一刻的  $\sigma$ ”与“当前 gradient”的加权平方平均, 权重由一个旋钮  $\alpha$  控制。<sup>8</sup>

逐步展开 RMSProp 的递推 ( $0 < \alpha < 1$ ):

$$\begin{aligned} \sigma_i^0 &= |g_i^0| \\ \sigma_i^1 &= \sqrt{\alpha(\sigma_i^0)^2 + (1 - \alpha)(g_i^1)^2} \\ \sigma_i^2 &= \sqrt{\alpha(\sigma_i^1)^2 + (1 - \alpha)(g_i^2)^2} \\ &\vdots \\ \sigma_i^t &= \sqrt{\alpha(\sigma_i^{t-1})^2 + (1 - \alpha)(g_i^t)^2} \end{aligned}$$

<sup>8</sup>视频画面时间区间: 00:19:07-00:21:21。

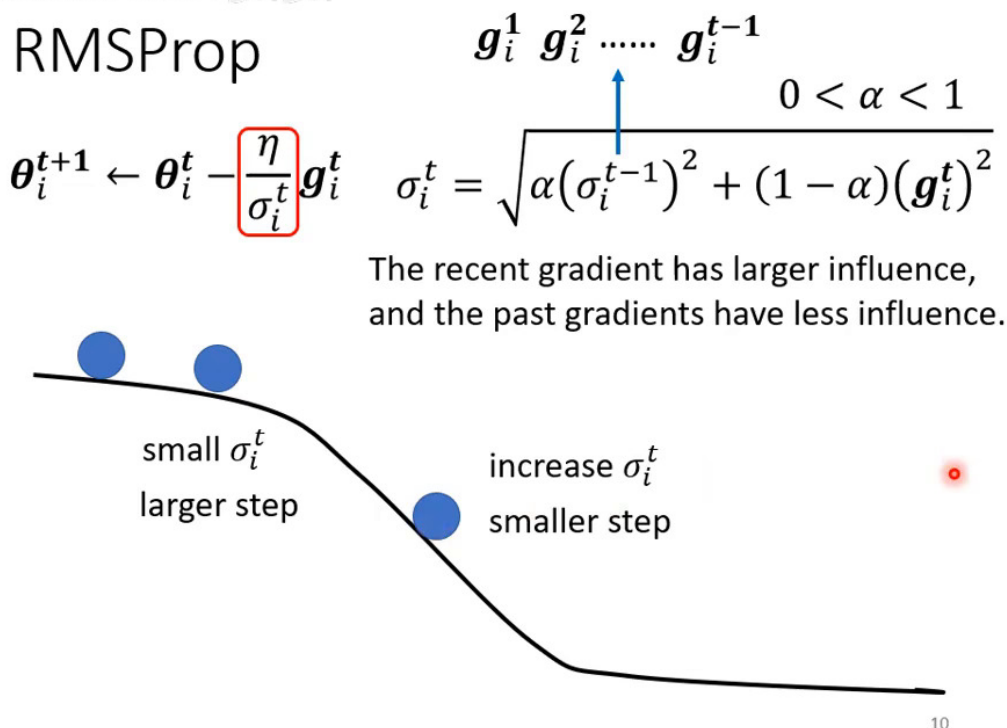


图 9: RMSProp 的一般式与直觉。 $\sigma_i^t = \sqrt{\alpha (\sigma_i^{t-1})^2 + (1 - \alpha) (g_i^t)^2}$  把“过去累积”与“当前梯度”按  $\alpha : (1 - \alpha)$  调和。由于当前 gradient 占了  $(1 - \alpha)$  的即时权重，一旦走进陡峭区段， $\sigma$  会迅速增大、有效步长立刻“踩刹车”；反之进入平缓区段， $\sigma$  又能较快回落、“加大油门”。学习率因此能灵敏地跟随 error surface 的陡缓变化。<sup>9</sup>

### RMSProp: 用 $\alpha$ 调节“记忆”与“灵敏”

递归式

$$\sigma_i^t = \sqrt{\alpha (\sigma_i^{t-1})^2 + (1 - \alpha) (g_i^t)^2}, \quad 0 < \alpha < 1.$$

旋钮  $\alpha$ （自己调的 hyperparameter）决定了“听过去”还是“听现在”：

- $\alpha$  越接近 1：越倚重历史累积  $\sigma_i^{t-1}$ ， $\sigma$  变化迟缓而平稳（更像 Adagrad）。
- $\alpha$  越接近 0：越倚重当前 gradient  $g_i^t$ ， $\sigma$  对陡缓变化反应灵敏，能快速“踩刹车/加油门”。

与 Adagrad 的刻板平均相比，RMSProp 让  $\sigma$  能动态跟随同一方向上的陡缓变化。

### RMSProp 的“身世”：没有论文的名方法

RMSProp 是个有趣的特例——它没有正式的论文出处。它最早出现在 Geoffrey Hinton 在 Coursera 开设的深度学习课程里，作为课堂内容被提出。要在文献里引用它，大家通常就引那门 Coursera 课程。尽管来历“草根”，它却是极其常用、效果稳健的自适应学习率方法。

<sup>9</sup>视频画面时间区间：00:21:21–00:23:15。

## 5 Adam: RMSProp + Momentum

到目前为止我们有了两条独立的改进线索：上一节的 **Momentum**（用过去所有 gradient 的方向累积出“惯性”，帮助冲过 critical point），和这一节的 **RMSProp**（用过去所有 gradient 的大小算出  $\sigma$ ，自适应地缩放步长）。把两者合在一起，就是当今最常用的优化器——**Adam**。



Original paper: <https://arxiv.org/pdf/1412.6980.pdf>

### Adam: RMSProp + Momentum

**Algorithm 1:** *Adam*, our proposed algorithm for stochastic optimization. See section 2 for details, and for a slightly more efficient (but less clear) order of computation.  $g_t^2$  indicates the elementwise square  $g_t \odot g_t$ . Good default settings for the tested machine learning problems are  $\alpha = 0.001$ ,  $\beta_1 = 0.9$ ,  $\beta_2 = 0.999$  and  $\epsilon = 10^{-8}$ . All operations on vectors are element-wise. With  $\beta_1^t$  and  $\beta_2^t$  we denote  $\beta_1$  and  $\beta_2$  to the power  $t$ .

**Require:**  $\alpha$ : Stepsize

**Require:**  $\beta_1, \beta_2 \in [0, 1)$ : Exponential decay rates for the moment estimates

**Require:**  $f(\theta)$ : Stochastic objective function with parameters  $\theta$

**Require:**  $\theta_0$ : Initial parameter vector

$m_0 \leftarrow 0$  (Initialize 1<sup>st</sup> moment vector)  $\rightarrow$  for momentum

$v_0 \leftarrow 0$  (Initialize 2<sup>nd</sup> moment vector)  $\rightarrow$  for RMSprop

$t \leftarrow 0$  (Initialize timestep)

**while**  $\theta_t$  not converged **do**

$t \leftarrow t + 1$

$g_t \leftarrow \nabla_{\theta} f_t(\theta_{t-1})$  (Get gradients w.r.t. stochastic objective at timestep  $t$ )

$m_t \leftarrow \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot g_t$  (Update biased first moment estimate)

$v_t \leftarrow \beta_2 \cdot v_{t-1} + (1 - \beta_2) \cdot g_t^2$  (Update biased second raw moment estimate)

$\hat{m}_t \leftarrow m_t / (1 - \beta_1^t)$  (Compute bias-corrected first moment estimate)

$\hat{v}_t \leftarrow v_t / (1 - \beta_2^t)$  (Compute bias-corrected second raw moment estimate)

$\theta_t \leftarrow \theta_{t-1} - \alpha \cdot \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon)$  (Update parameters)

**end while**

**return**  $\theta_t$  (Resulting parameters)

11

图 10: Adam = RMSProp + Momentum。投影片直接贴出原始论文 (*Adam: A Method for Stochastic Optimization*, arXiv:1412.6980) 的 Algorithm 1 伪代码：其中  $m_t$  这一路是 **Momentum**（累积梯度方向）， $v_t$  这一路就是 **RMSProp** 式的梯度平方累积（对应  $\sigma$ ），再各自做偏差校正后组合更新。**Adam** 是 **PyTorch** 里的预设优化器之一，多数情况下直接拿来用、连超参数都用预设值，效果就已经很好。<sup>10</sup>

#### Adam 的一句话理解

**Adam** = **Momentum**（决定“往哪个方向走”）+ **RMSProp**（决定“这个方向上步子该多大”）。它同时吸收了两条改进：用 momentum 平滑出更可靠的前进方向、对抗 critical point；用 RMSProp 式的  $\sigma$  为每个参数自适应缩放步长、化解陡缓两难。论文里另含偏差校正等细节，但工程上你几乎不必操心——**PyTorch** 一行 `torch.optim.Adam` 配预设超参数，就是绝大多数任务的稳妥起点。

<sup>10</sup> 视频画面时间区间：00:23:15–00:24:32。

## 6 学习率调度：Decay 与 Warm Up

自适应的  $\sigma$  已经解决了“不同方向”的步长问题。但还有一个维度没动过：分子上的  $\eta$  始终是个固定常数。把 Adagrad 放回最初那个凸面实验，就会暴露出只调  $\sigma$  仍不够的现象。

### 6.1 动机：Adagrad 会“喷出去”再自我修正

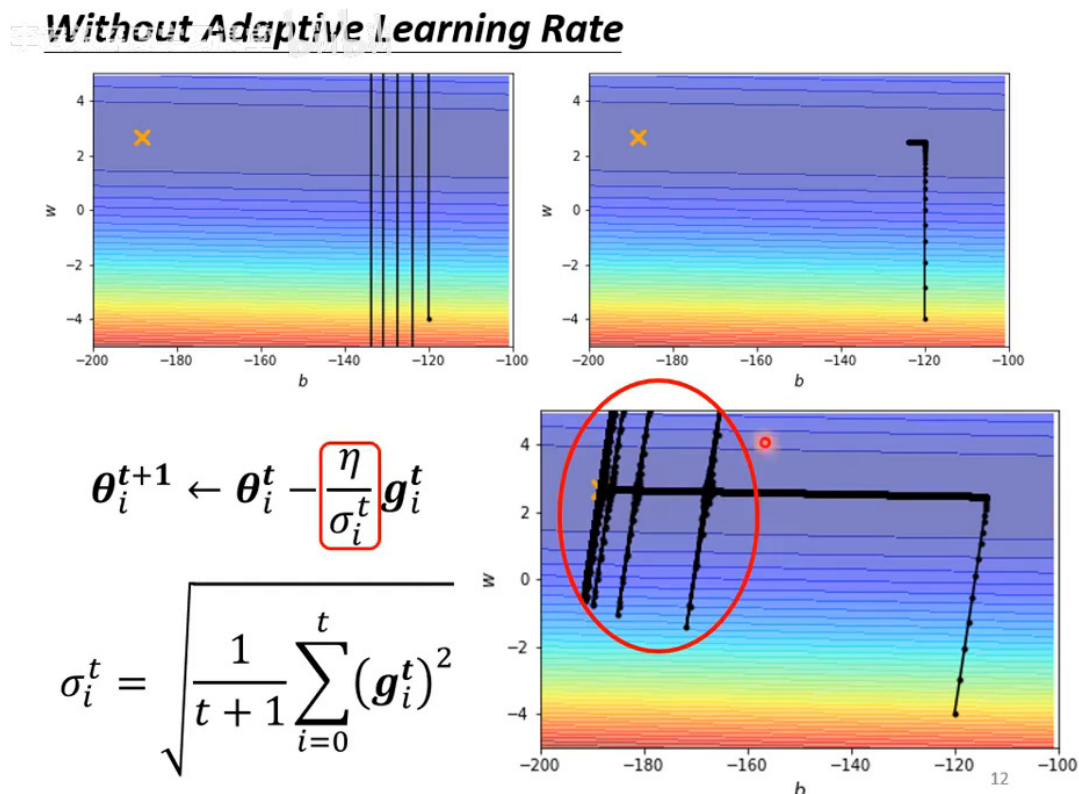


图 11: 回到凸面实验。上方两小图重述固定学习率的失败（左  $\eta = 10^{-2}$  纵向剧烈震荡；右  $\eta = 10^{-7}$  走一小段便几乎停住）。下方大图（红椭圆圈出关键现象）是用了 Adagrad 之后的轨迹：参数先顺利沿陡峭方向滑到谷底、再沿平缓方向左行；但平缓方向走久了、历史梯度的平方累积使  $\sigma$  变得很小，有效步长骤然暴增，于是猛地“喷”出几道纵向尖峰；一“喷”进陡峭区，gradient 立刻变大、 $\sigma$  随之回升，步长又被拉回——如此自我修正，最终仍能抵达终点，只是过程颇为“抖动”。<sup>11</sup>

这个“喷出去”说明：光让  $\sigma$  自适应还不够稳，我们最好再让分子上的  $\eta$  也随时间变化，记作  $\eta^t$ 。这就是 **Learning Rate Scheduling (学习率调度)**：

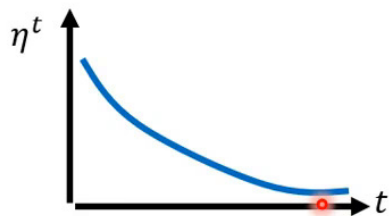
$$\theta_i^{t+1} \leftarrow \theta_i^t - \frac{\eta^t}{\sigma_i^t} g_i^t.$$

<sup>11</sup> 视频画面时间区间：00:24:32–00:27:21。

## 6.2 Learning Rate Decay: 越接近终点，步子越小

### Learning Rate Scheduling

$$\theta_i^{t+1} \leftarrow \theta_i^t - \frac{\eta^t}{\sigma_i^t} g_i^t$$



### Learning Rate Decay

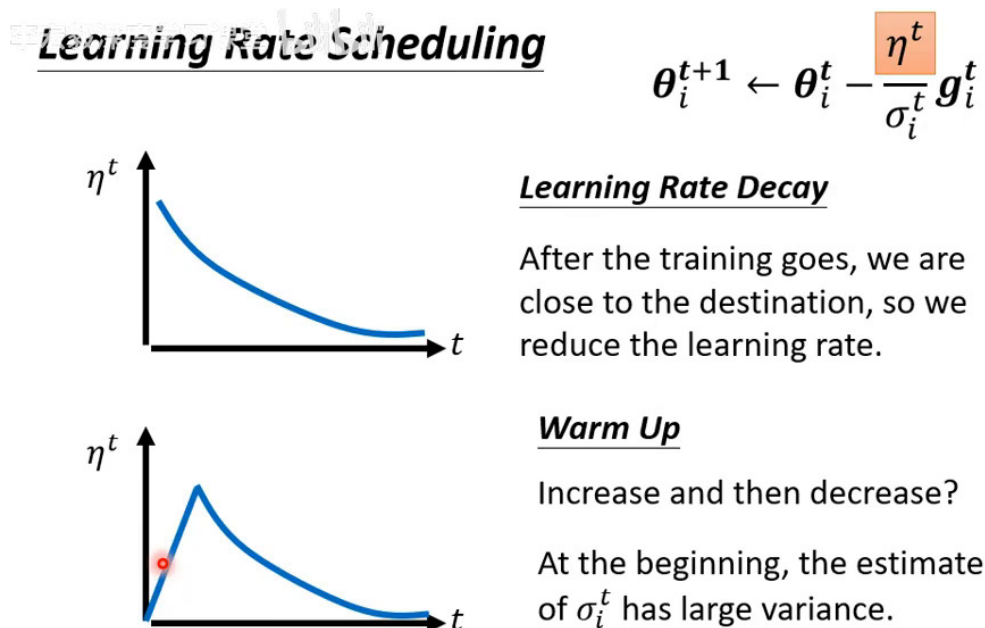
As the training goes, we are closer to the destination, so we reduce the learning rate.

13

图 12: 最常见的调度是 **Learning Rate Decay** (学习率衰减): 让  $\eta^t$  随训练进行而逐渐减小 (曲线单调下降)。直觉是——*As the training goes, we are closer to the destination, so we reduce the learning rate* (训练越久、离终点越近, 就该把步子收小, 以免在终点附近冲过头)。把  $\eta^t$  这样递减, 能直接缓解上图 Adagrad “喷出去” 的抖动。<sup>12</sup>



### 6.3 Warm Up: 先升后降的反直觉技巧



16

图 13: 另一种著名调度是 **Warm Up**:  $\eta^t$  的曲线先由小变大、到达峰值后再慢慢变小 (*Increase and then decrease?*)。它看似反直觉, 却在许多知名模型里都是标配。投影片下方点明了**为什么需要 warm up**: *At the beginning, the estimate of  $\sigma_i^t$  has large variance*——训练初期  $\sigma$  是用很少的 gradient 统计出来的, 估计很不准 (方差大), 所以先用较小的学习率“探路”、等  $\sigma$  的统计逐渐可靠后再把学习率拉上来。<sup>13</sup>

#### 为什么 Warm Up 有道理: $\sigma$ 是个“需要时间才准”的统计量

回想  $\sigma_i^t$  是对历来 gradient 大小的统计估计。训练刚开始时, 手头只有寥寥几个 gradient,  $\sigma$  的估计方差很大、很不可靠。此时若直接放开大学习率, 很容易被一个失真的  $\sigma$  带偏、走到 error surface 上奇怪的地方。Warm Up 的做法是: 先用小学习率走一阵, 一边收集 gradient、让  $\sigma$  的统计变准, 再逐步升到正常学习率, 之后才转入常规的 decay。

#### Warm Up 的“江湖地位”: 从 ResNet 到 Transformer

Warm Up 绝非偏方, 而是诸多里程碑模型的标准配置: 残差网络 **Residual Network (ResNet, arXiv:1512.03385)** 的训练用到它, **Transformer** 原始论文里也明确使用了 warm up。后来更有专门改进自适应学习率初期不稳定的工作, 如 **RAdam (Rectified Adam)**, 本质上也是在处理“初期  $\sigma$  估计不准”这个同源问题。换句话说, warm up 是工程界反复验证过的有效技巧。

<sup>12</sup> 视频画面时间区间: 00:27:21–00:28:49。

<sup>13</sup> 视频画面时间区间: 00:28:49–00:34:09。

## 7 优化的全景总结 (Summary of Optimization)

把这一节（自适应学习率、调度）与上一节（momentum）的所有改进收拢到一起，就得到现代优化器的“全家福”更新式。

中发数据科学学习课堂 Bilibili

### Summary of Optimization

#### (Vanilla) Gradient Descent

$$\theta_i^{t+1} \leftarrow \theta_i^t - \eta g_i^t$$

#### Various Improvements

$$\theta_i^{t+1} \leftarrow \theta_i^t - \frac{\eta^t}{\sigma_i^t} m_i^t$$

Learning rate scheduling

Momentum: weighted sum of the previous gradients

Consider direction

root mean square of the gradients

only magnitude

17

图 14: 优化方法的全景总结。最朴素的 (Vanilla) Gradient Descent 是  $\theta_i^{t+1} \leftarrow \theta_i^t - \eta g_i^t$ ；各项改进汇合后变成  $\theta_i^{t+1} \leftarrow \theta_i^t - \frac{\eta^t}{\sigma_i^t} m_i^t$ 。三个部件各司其职： $m_i^t$  是 **Momentum**（过去所有 gradient 的加权和，consider direction，决定方向）； $\sigma_i^t$  是 **Root Mean Square**（过去所有 gradient 大小的均方根，only magnitude，决定步长缩放）； $\eta^t$  是 **Learning Rate Scheduling**（随时间安排的整体步长）。<sup>14</sup>

#### 现代优化器的统一更新式

$$\theta_i^{t+1} \leftarrow \theta_i^t - \frac{\eta^t}{\sigma_i^t} m_i^t$$

- $m_i^t$  (**Momentum**, 方向): 过去所有 gradient 的加权和, 是个向量, 决定“往哪个方向走”。
- $\sigma_i^t$  (**Root Mean Square**, 大小): 过去所有 gradient 的均方根, 是个正的标量, 决定“这个方向的步子该缩放多少”。
- $\eta^t$  (**Scheduling**): 随训练时刻  $t$  安排的整体学习率 (如 decay、warm up)。

<sup>14</sup> 视频画面时间区间: 00:34:09–00:36:42。

$m_i^t$  与  $\sigma_i^t$  都用过去的 gradient，会不会互相抵消？

课堂上有同学问：分子的 momentum  $m_i^t$  和分母的  $\sigma_i^t$  都来自“过去所有 gradient”，岂不是会约掉、白忙一场？**答案是不会**，因为二者用 gradient 的方式根本不同：

- $m_i^t$  是向量和，看方向——方向相反的 gradient 会相互抵消（一上一下相加趋于零）。
- $\sigma_i^t$  是平方和开根，只看大小——gradient 先平方，正负都变正，**绝不会抵消**，只会越累越大。

所以即便分子分母都“装了”同一批 gradient，一个算方向、一个算大小，处理逻辑互不干涉，**不会相消**。这正是投影片上 *consider direction* 与 *only magnitude* 两个注解的用意。

## 8 总结与延伸

本节从一个被忽视的事实出发——**loss 卡住时往往 gradient 并不小、根本没到 critical point**——一步步推出了“自动调整学习率”这条完整的改进链：从客制化  $\sigma$ 、到 Adagrad、RMSProp、Adam，再到  $\eta^t$  的 Decay 与 Warm Up，最终汇成现代优化器的统一更新式。

### 8.1 核心要点提炼

自适应学习率：四个关键认知

1. **停滞  $\neq$  critical point**：loss 不降时 gradient 常常还很大，真凶多是“步伐没调好”——在峡谷两壁间反复横跳。
2. **固定  $\eta$  顾此失彼**：陡峭方向要小步、平缓方向要大步，单一学习率做不到，必须**因参数、因时刻定制**。
3. **用  $\sigma$  缩放步长**：以 gradient 大小的统计量  $\sigma_i^t$  当分母——梯度大则步小、梯度小则步大。Adagrad 用整段历史的 RMS；RMSProp 用带  $\alpha$  的递归式，能**动态跟随陡缓变化**；**Adam = RMSProp + Momentum**。
4. **再调度  $\eta^t$** ：Learning Rate Decay 让步子随训练收小；Warm Up 先升后降，化解**初期  $\sigma$  估计方差大的不稳**。

从 Vanilla 到现代优化器的演化

$$\begin{aligned} \text{Vanilla GD:} \quad & \theta_i^{t+1} \leftarrow \theta_i^t - \eta g_i^t \\ + \text{客制化 / Adagrad:} \quad & \theta_i^{t+1} \leftarrow \theta_i^t - \frac{\eta}{\sigma_i^t} g_i^t \quad (\sigma \text{ 取历史 RMS}) \\ + \text{RMSProp:} \quad & \sigma_i^t = \sqrt{\alpha(\sigma_i^{t-1})^2 + (1-\alpha)(g_i^t)^2} \quad (\sigma \text{ 可动态跟随}) \\ + \text{Momentum = Adam:} \quad & \text{分子 } g_i^t \text{ 换成方向累积 } m_i^t \\ + \text{Scheduling:} \quad & \theta_i^{t+1} \leftarrow \theta_i^t - \frac{\eta^t}{\sigma_i^t} m_i^t \quad (\eta^t: \text{ decay / warm up}) \end{aligned}$$

## 8.2 本节关键概念清单

术语对照表

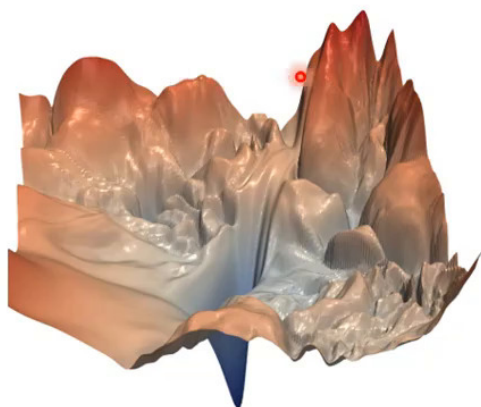
| 概念      | 英文                         | 说明                               |
|---------|----------------------------|----------------------------------|
| 自适应学习率  | Adaptive Learning Rate     | 让每个参数、每个时刻有专属步长                  |
| 凸误差曲面   | Convex Error Surface       | 无 local minima / saddle 的碗状曲面    |
| 客制化步长   | Parameter-dependent $\eta$ | 把 $\eta$ 换成 $\eta/\sigma_i^t$    |
| 均方根     | Root Mean Square (RMS)     | 历史 gradient 平方平均再开根, 即 $\sigma$  |
| Adagrad | Adagrad                    | $\sigma$ 取整段历史 RMS 的方法           |
|         | RMSProp                    | $\sigma$ 用带 $\alpha$ 的递归式, 可动态跟随 |
| 平滑系数    | $\alpha$                   | 调和“历史累积”与“当前梯度”的权重               |
| Adam    | Adam                       | RMSProp + Momentum; PyTorch 预设   |
| 学习率衰减   | Learning Rate Decay        | $\eta^t$ 随训练逐渐减小                 |
| 预热      | Warm Up                    | $\eta^t$ 先升后降, 缓解初期 $\sigma$ 不准  |
| 学习率调度   | LR Scheduling              | 让 $\eta$ 随时刻 $t$ 变化为 $\eta^t$    |
| 动量      | Momentum $m_i^t$           | 过去梯度的方向加权和 (向量)                  |

## 8.3 承上启下

- **承上**: 上一节用 small batch 的噪声与 momentum 的惯性对抗 critical point; 本节进一步指出, 很多停滞压根不是 **critical point**, 而是学习率不当——于是给出了自适应学习率这一整套“治本”方案。至此, 所有改进都集中在“怎么在固定的 error surface 上走得更好”。
- **启下**: 但还有一条更釜底抽薪的思路——与其想办法在崎岖的 error surface 上跋涉, 不如直接把 error surface 改造得平坦好走。

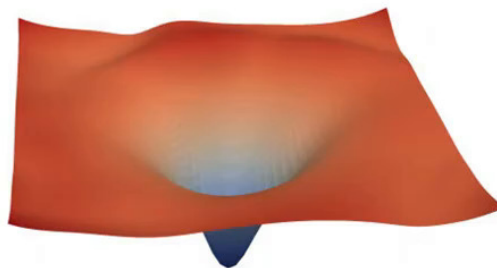
## Next Time

Source of image: <https://arxiv.org/abs/1712.09913>



Better optimization strategies:  
If the mountain won't move,  
build a road around it.

Next time



Can we change the error  
surface?  
Directly move the mountain!

19

图 15: 下回预告 (图片出处: arXiv:1712.09913)。左侧是崎岖陡峭、沟壑纵横的 error surface, 右侧是被“抚平”成的平滑碗状曲面。老师用两句话点题: *If the mountain won't move, build a road around it* (山不过来, 就修条路绕过去) 与 *Can we change the error surface? Directly move the mountain!* (能不能干脆改造 error surface? 直接把山移走!) ——下一讲将讨论如何主动改造 error surface (如 Batch Normalization 等), 让训练从源头变得容易。<sup>15</sup>

---

本节完 · 下一节: 如何改造 error surface ——让训练从源头变容易

课程视频: <https://www.bilibili.com/video/BV1TAtwzTE1S?p=6>

---

<sup>15</sup> 视频画面时间区间: 00:36:42–00:37:39。